

Computer Architecture

Lab 2: Out-of-Order RISC-V Processor - Reorder Buffer

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

revision: 9-29-2026

In this lab, you will complete the scoreboard and implement a reorder buffer (ROB) for an out-of-order, single-issue RISC-V processor. The provided processor framework includes an incomplete scoreboard and a dummy ROB implementation. Your objectives are to complete these components and create appropriate test cases.

This lab will provide you with experience in:

- The fundamentals of reorder buffers and their role in out-of-order execution.
- How to design and implement a scoreboard for dependency tracking, hazard detection, and operand bypassing.
- How to design and implement in-order commitment logic within an out-of-order processor.

1 Reminder

- **Submission Deadline:** 10/12 (Mon) 11:59 p.m.
- Please refer to Lab 0 for the academic integrity requirements.
- **No sharing or distribution of lab materials is allowed.**

2 Setup

2.1 Getting Started

After obtaining the Lab 2 materials, extract them using the following commands:

```
% tar -xf lab2.tar
% cd lab2
% export LAB2_ROOT=$PWD
```

Inside the lab root directory, you'll find these subdirectories, each serving a specific purpose:

- **build:** Makefile and compiled code.
- **riscv000:** Out-of-Order RISC-V processor source code (includes a placeholder reorder buffer).
- **tests:** Assembly test build system.
 - **riscv:** RISC-V assembly tests.
 - **scripts:** Utility scripts for the build system.
- **ubmark:** Benchmarks for evaluation.

- vc: Additional Verilog components.

Most directories remain unchanged from the previous lab. The new directory, `riscvooo`, contains the OoO processor framework, including an incomplete scoreboard and a dummy ROB implementation.

2.2 Building the Project

The commands below describe the build and test procedure. As usual, we'll start by compiling the reference processors and running the RISC-V assembly tests:

```
% cd $LAB2_ROOT/tests
% mkdir build && cd build
% ../configure --host=riscv32-unknown-elf
% make && ../convert
```

```
% cd $LAB2_ROOT/build
% make
% make check
% make check-asm-riscvooo
% make check-asm-rand-riscvooo
```

Then run the benchmark:

```
% cd $LAB2_ROOT/ubmark
% mkdir build && cd build
% ../configure --host=riscv32-unknown-elf
% make && ../convert
```

```
% cd $LAB2_ROOT/build
% make
% make check
% make run-bmark-riscvooo
% make run-bmark-rand-riscvooo
```

3 Out-of-Order Processor Description

The out-of-order (OoO) processor in Lab 2 is derived from the `riscvlong` microarchitecture from Lab 1, so most of the design should feel familiar. After you complete the scoreboard, the processor with the dummy ROB operates as a simplified I2O2 processor: `fetch`, `decode`, and `issue` occur in order, while `execution`, `writeback`, and `commit` are out of order. You will then implement and integrate the ROB to convert it into an I2OI processor. Key differences from the lecture model include:

- The pipeline includes three pipes: The MUL pipe handles `muldiv` operations and takes four cycles to execute. The MEM pipe handles memory accesses and takes two cycles to execute. The ALU pipe handles all other operations and takes one cycle to execute.
- Memory writes occur directly in the M stage without the need for a store buffer. This simplifies the design in this lab by ensuring that memory operations execute sequentially and without exception handling.
- Unlike some configurations discussed in lecture, this processor includes a **single** (architectural) register file. Instead of using a physical register file or storing uncommitted data directly in the ROB, we have a dedicated buffer within the datapath to hold data that has been written back but not yet committed.
- The reorder buffer in this lab does not include speculative bits, as speculative execution is not needed.

In this lab, you will work with a reorder buffer containing 16 entries, sufficient for the maximum expected in-flight instructions. The structure of the ROB is illustrated in Table 1.

Table 1: Reorder Buffer Contents

Entry	Valid	Pending	Physical Register	Data
0	1	1	0	0x00000000
1	1	0	3	0xDEADBEEF
...	...	...	...	...
15	1	1	0	0x00000000

Each entry in the ROB uses a **valid bit** to indicate if it is active; this bit is set when the entry is allocated and cleared when the entry is committed. The **pending bit** is set at allocation and cleared when the result is written back to the ROB. The **physical register bits** store the destination register for the entry in the architectural register file.

Two 4-bit pointers manage the ROB: the **head pointer**, which indicates the next entry to be committed, and the **tail pointer**, which indicates the next available entry for allocation. The tail pointer increments with each new allocation, and the head pointer increments upon commitment. Although capacity issues are unlikely in this lab, ensure boundary checks are in place to stall the processor if the ROB becomes full.

Instruction data is stored in the provided array, `rob_data`, located in the datapath. The ROB generates the control signals for this array.

Finally, speculative bits are not needed in this lab's simplified design. In cases where a conditional branch in the execute stage (X) could impact an instruction in decode (D), the ROB simply skips allocation if the instruction in D is squashed. While this would increase cycle time in a real processor, it is acceptable for this lab's simulation.

4 Building the Scoreboard

Before building the reorder buffer, you will first complete the scoreboard implementation in `riscv000-CoreScoreboard.v`. The scoreboard is responsible for tracking register dependencies and determining whether an instruction can safely proceed through the pipeline.

Since instructions may have different execution latencies depending on the functional unit they use, the scoreboard must keep track of the status of each architectural register and determine when a source operand becomes available.

The scoreboard maintains the following information for each register:

- **pending**: Indicates whether the register is waiting for a value produced by an in-flight instruction.
- **functional_unit**: Records which functional unit is producing the register value.
- **reg_latency**: Tracks the progress of the instruction producing the register value.
- **reg_rob_slot**: Records the ROB slot associated with the instruction that writes to the register.

The processor contains three functional units:

- **ALU**: Handles arithmetic and logical instructions.
- **MEM**: Handles memory operations.
- **MUL**: Handles multiply and divide operations.

Because these functional units have different execution latencies, the scoreboard must correctly track when their results become available and whether an instruction can obtain its operands through bypassing.

The scoreboard provides the following main functions:

- **Dependency Tracking:** Determine whether the source registers of an instruction are available.
- **Hazard Detection:** Stall an instruction when a required operand is not ready or when a writeback conflict occurs.
- **Latency Tracking:** Track the progress of instructions through different functional units while considering pipeline stalls.
- **Operand Bypassing:** Select the appropriate bypass source when an operand is available from a later pipeline stage.
- **ROB Tracking:** Associate each destination register with its allocated ROB slot for ROB bypassing and commit handling.

The input signals used for dependency and hazard detection include:

- `src0` and `src1`: Specify the source registers of the current instruction.
- `src0_en` and `src1_en`: Indicate whether each source register is used.
- `dst`: Specifies the destination register.
- `dst_en`: Indicates whether the instruction writes to a destination register.
- `func_unit`: Specifies which functional unit executes the instruction.
- `latency`: Specifies the expected execution latency of the instruction.
- `stalls`: Indicates the current pipeline stall conditions.

The scoreboard generates the following control signals:

- `stall_hazard`: Indicates whether the instruction must stall due to a dependency or resource conflict.
- `src0_byp_mux_sel` and `src1_byp_mux_sel`: Select the bypass source for each source operand.
- `src0_byp_rob_slot` and `src1_byp_rob_slot`: Specify the ROB slot associated with each source register.
- `wb_mux_sel`: Selects which functional unit uses the writeback path.
- `stall_wb_hazard_X` and `stall_wb_hazard_M`: Indicate writeback conflicts between functional units.

Complete the scoreboard logic in `riscv000-CoreScoreboard.v` without changing the provided module interface. Your implementation should correctly track register dependencies, update instruction latency according to pipeline stalls, detect hazards, and generate the required bypass and writeback control signals.

After completing the scoreboard, run the provided test suites and verify that the processor correctly handles register dependencies and operand bypassing. Some cases related to out-of-order commit may still fail at this stage. These cases will be addressed after implementing the reorder buffer in the next section.

5 Building the ROB

Using the scoreboard you completed in Section 4, build the I2O2 processor with the dummy ROB and run the included RISC-V test suites. **Interestingly, even though the I2O2 processor may commit values out of order, it still passes all provided tests!** This suggests that the test suite is not exhaustive, as you may have noticed in previous labs.

Your first task is to design an assembly test that reveals the flaw in the I2O2 processor's commit behavior. Specifically, construct a scenario where out-of-order commits lead to incorrect program execution, and verify this behavior on the I2O2 processor. (Hint: such a test should be straightforward to create.)

Once you've confirmed the need for in-order commit and have created a suitable assembly test (or, ideally, multiple tests), it's time to build the reorder buffer (ROB).

The ROB uses two integers defined in `riscv000-InstMsg.v`:

- ``SLOTS` - The number of ROB slots.
- ``LOG_S` - The ceiling of  $\log(\texttt{`SLOTS})$.

The ROB consists of three main port groups, each serving a distinct function:

- **Alloc** - Instructions in the decode stage must be assigned a new ROB entry before they execute.
- **Fill** - Instructions in the writeback stage write their data into the ROB through the fill ports.
- **Commit** - One instruction may be committed per cycle, provided all necessary values are ready.

The ports of the ROB are described below:

- `rob_alloc_req_val`: Indicates if an allocation request is valid in the current cycle.
- `rob_alloc_req_rdy`: Indicates if the ROB is ready to accept an allocation request.
- `rob_alloc_req_preg`: Specifies the physical destination register for the incoming instruction.
- `rob_alloc_resp_slot`: The slot number in the ROB assigned to the new instruction.
- `rob_fill_val`: Indicates if a value is being written back into the ROB during the current cycle.
- `rob_fill_slot`: Specifies the slot being written back during the current cycle.
- `rob_commit_wen`: Indicates if an entry is being committed to the register file in this cycle.
- `rob_commit_slot`: Specifies which slot is committed, used as an index for the `rob_data` array.
- `rob_commit_rf_waddr`: Specifies the physical register address to commit.

Implement the ROB logic by modifying only the `riscv000-CoreReorderBuffer.v` file. After completing the ROB, update the register file module in `riscv000-CoreDpath.v` to retrieve values from the ROB instead of directly from the writeback stage. **Then, enable (uncomment) the lines in `riscv000-CoreScoreboard.v` that allow for bypassing from the ROB.** Finally, rerun the provided test suites to ensure they pass, and confirm that your custom test case also now passes correctly.

6 Testing Methodology

We have created the blank test file in `/lab2/tests/riscv`, you need to fill it and modify `/lab2/tests/riscv/riscv.mk` and `/lab2/build/Makefile`, then, execute, and include in your submission at least the following tests:

(Please make sure that each of your tests is less than 30 instructions)

- `riscv-test1.S`: A test that fails due to out-of-order commits but passes with the reorder buffer (ROB) implemented (see Section 5).
- `riscv-test2.S`: A test case requiring a value to be bypassed from the ROB (i.e., bypassing a value that has been written back but not yet committed).
- `riscv-test3.S`: A write-after-write (WAW) scenario that executes correctly on both the original and the final processor. (Hint: Consider the time elapsed between the two writes.)
- `riscv-test4.S`: A scenario in which the `riscvlong` processor achieves a higher IPC than the completed `riscv000` processor. (Hint: What features are present in `riscv000` but not in `riscvlong`?)
- `riscv-test5.S`: A test which will use as many ROB slots as your design can.
- Any additional tests you find interesting.

7 Evaluation (Optional)

The following evaluation tasks are optional and are not required for completing this lab.

- **Performance Comparison:** Compare the `riscv000` processor with the `riscvlong` processor. For each benchmark, record the total cycle count and IPC (instructions per cycle). You can analyze:
 - When and why one design outperforms the other.
 - Situations where performance differences are minimal.
 - Which terms in the Iron Law of Processor Performance are affected.
- **Reorder Buffer (ROB) Analysis:** Explore how performance scales with different ROB sizes. Vary the number of ROB slots (2–16) and measure the cycle count and IPC for `riscv-test5.S`. Discuss:
 - How performance changes as the ROB size increases.
 - The minimum number of ROB slots needed for this lab and your reasoning.

8 Submission

8.1 Modified Files

You will modify the following files for this lab:

- `riscv000-CoreDpath.v`: Make minor edits as indicated by the comments in the source code.
- `riscv000-CoreScoreboard.v`: Complete the scoreboard implementation.
- `riscv000-CoreReorderBuffer.v`: Implement your reorder buffer design here.
- `riscv000-InstMsg.v`: Adjust the parameter that specifies the number of ROB slots.
- `riscv-test1.S` to `riscv-test5.S`: See Section 6. ""

8.2 Deliverables

Submit a `.tar.gz` file of your working directory, keeping the original structure intact. All source files should be in `$LAB2_ROOT/riscv000/` or in the `$LAB2_ROOT/tests/` directory. Be sure to clean up generated waveforms or compiled code by running the following commands:

```
% cd $LAB2_ROOT/build
% make clean
% cd $LAB2_ROOT/tests
% rm -rf build
% cd $LAB2_ROOT/ubmark
% rm -rf build
```

Create a tarball of your completed lab with the following commands:

```
% cd $LAB2_ROOT
% cd ..
% tar -cvzf student_id-lab2.tar.gz lab2
```

8.3 Submission Instructions

- Keep your code in the `lab2` folder. If the code is not in this tarball, we cannot grade it.
- Submit the tarball via e3.
 - `student_id-lab2.tar.gz`

9 Grading Rubric

- **Code (90%)**
 - ROB
 - Scoreboard
- **Test case(10%)**
 - `riscv-test1.S` to `riscv-test5.S`

10 Tips

- Develop incrementally—code and test small sections at a time to avoid overwhelming debugging.
- Always draw the hardware design before coding, ensuring clear interaction between control logic and the datapath.
- Modify the control unit as needed—it's a starting point for your own logic and signals.
- Start early and run simulations regularly to catch issues early in the process.